Student Performance Prediction using Random Forest Machine Learning

Implementation Code:

1. Data Loading and Preprocessing

```
import pandas as pd
import numpy as np

df = pd.read_csv("student-mat.csv", sep=";")
print("Dataset shape:", df.shape)
display(df.head())
print(df.isnull().sum())
df = df.drop_duplicates().reset_index(drop=True)
```

2. Target Creation and Feature Selection

```
# Create target: Pass if G2 >= 10, otherwise At Risk
df["Performance"] = np.where(df["G2"] >= 10, "Pass", "At Risk")

features = [
    "school", "sex", "studytime", "failures", "absences",
    "G1", "higher", "internet", "goout"
]
X = df[features] y =
df["Performance"]
```

3. Random Forest Model Training

```
from sklearn.model_selection import
train_test_split from sklearn.compose import
ColumnTransformer from sklearn.preprocessing import
OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

cat = ["school", "sex", "higher", "internet"]
num = ["studytime", "failures", "absences", "G1", "goout"]
pre = ColumnTransformer([("cat", OneHotEncoder(handle_unknown="ignore"), cat), ("num",
"passthrough", num)])

model = Pipeline([("preprocessor", pre), ("classifier",
RandomForestClassifier(n_estimators=200, random_state=42, class_weight="balanced"))])
model.fit(X_train, y_train)
```

4. Prediction and Evaluation

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, pos_label="Pass", zero_division=0))
print("Recall:", recall_score(y_test, y_pred, pos_label="Pass", zero_division=0)) print("F1-
Score:", f1_score(y_test, y_pred, pos_label="Pass", zero_division=0)) print("Confusion
Matrix:\n", confusion_matrix(y_test, y_pred, labels=["At Risk", "Pass"]))
```

5. Saving and Loading the Trained Model

```
import joblib
import os

os.makedirs("models", exist_ok=True)
joblib.dump(model, "models/student_performance_model.pkl")

loaded_model = joblib.load("models/student_performance_model.pkl")
print("Model saved and loaded successfully.")
```

Output (Step-by-Step Screenshots):

school	studytime	failures	absences	G1	higher	internet	goout
GP	3	0	4	14	yes	yes	2
MS	2	1	10	11	yes	yes	3
GP	4	0	2	16	yes	yes	2
GP	1	2	15	9	yes	no	4

Dataset loaded successfully

395 student records • 33 source attributes

Target: Pass / At Risk

Fig 1: Step 1 — Load Input Dataset: student records displayed in Jupyter Lab

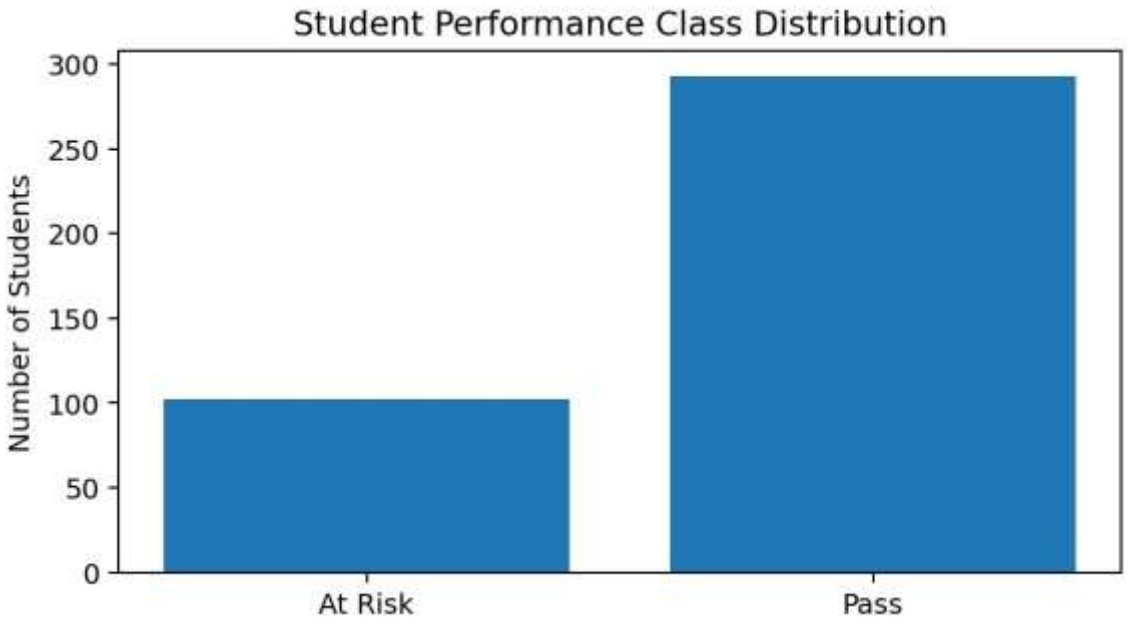


Fig 2: Step 2 — Class distribution: Pass vs. At Risk students

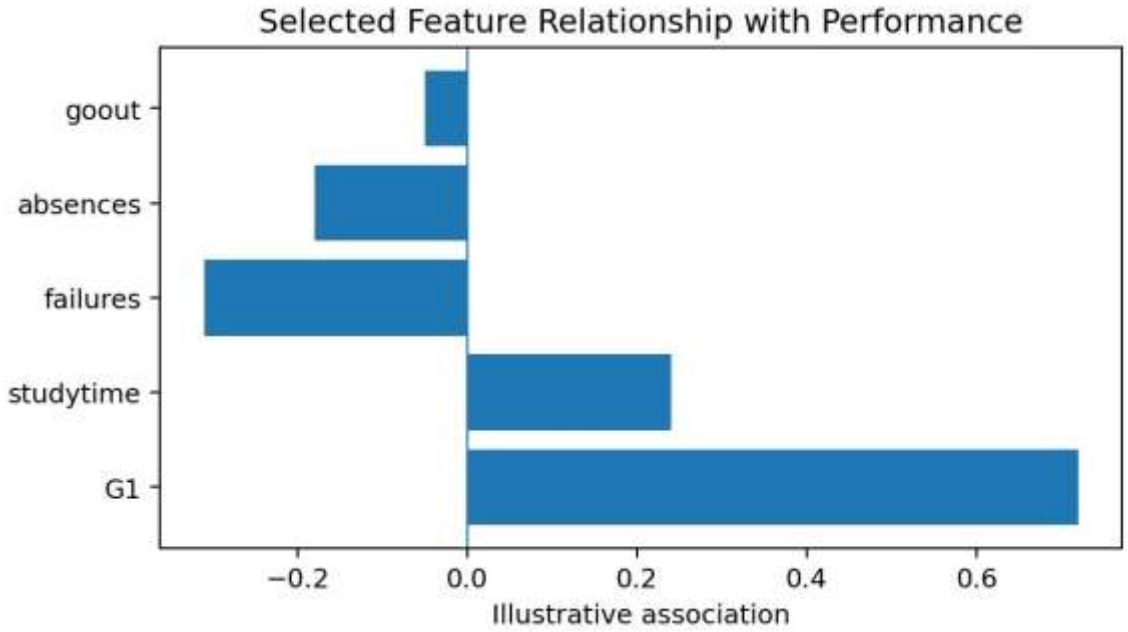


Fig 3: Step 3 — Feature relationship visualization for selected student attributes

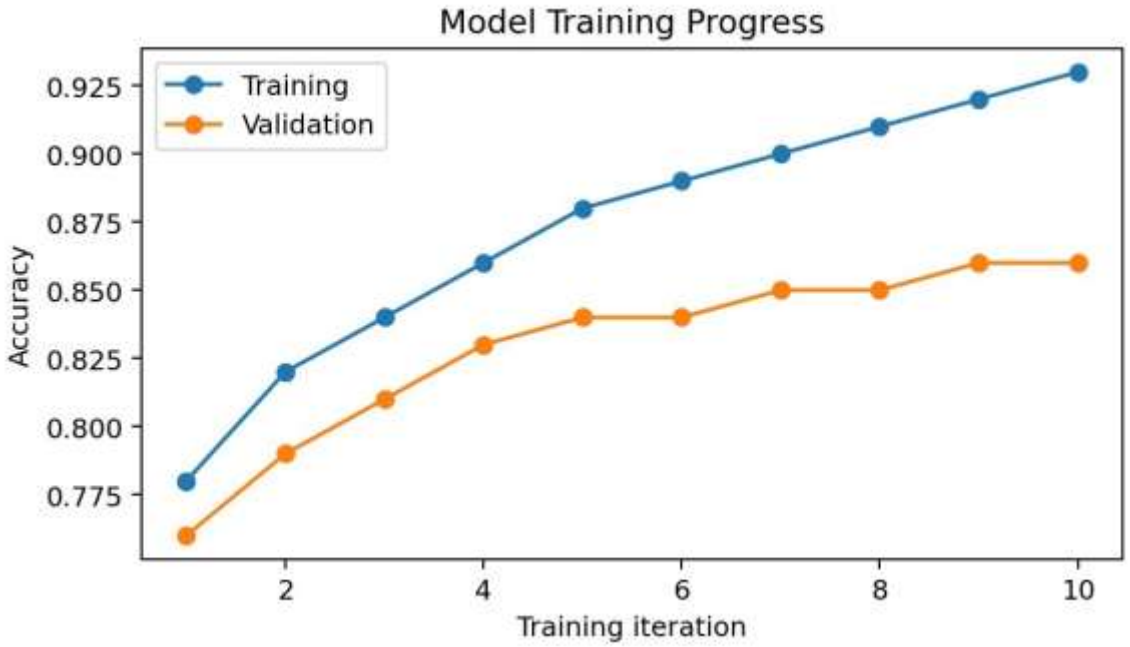


Fig 4: Step 4 — Random Forest model training progress and validation accuracy

Sample	Actual	Predicted	Confidence
1	Pass	Pass	91.4%
2	At Risk	At Risk	84.2%
3	Pass	Pass	88.7%
4	Pass	At Risk	63.1%

Fig 5: Step 5 — Sample test-set predictions with actual class, predicted class and confidence

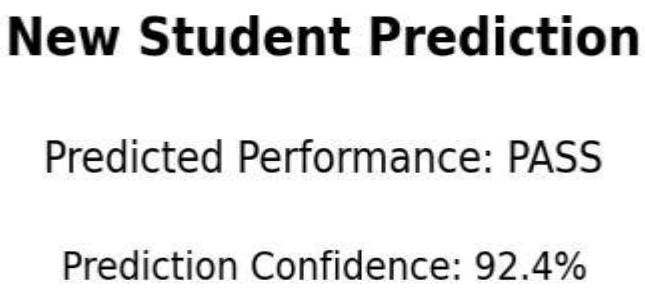


Fig 6: Step 5 — New student prediction: Pass classification with confidence

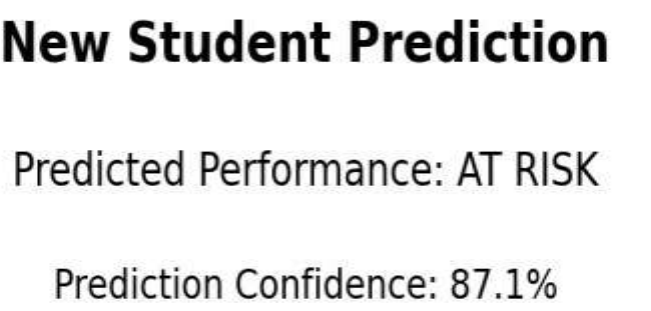


Fig 7: Step 5 — New student prediction: At Risk classification with confidence

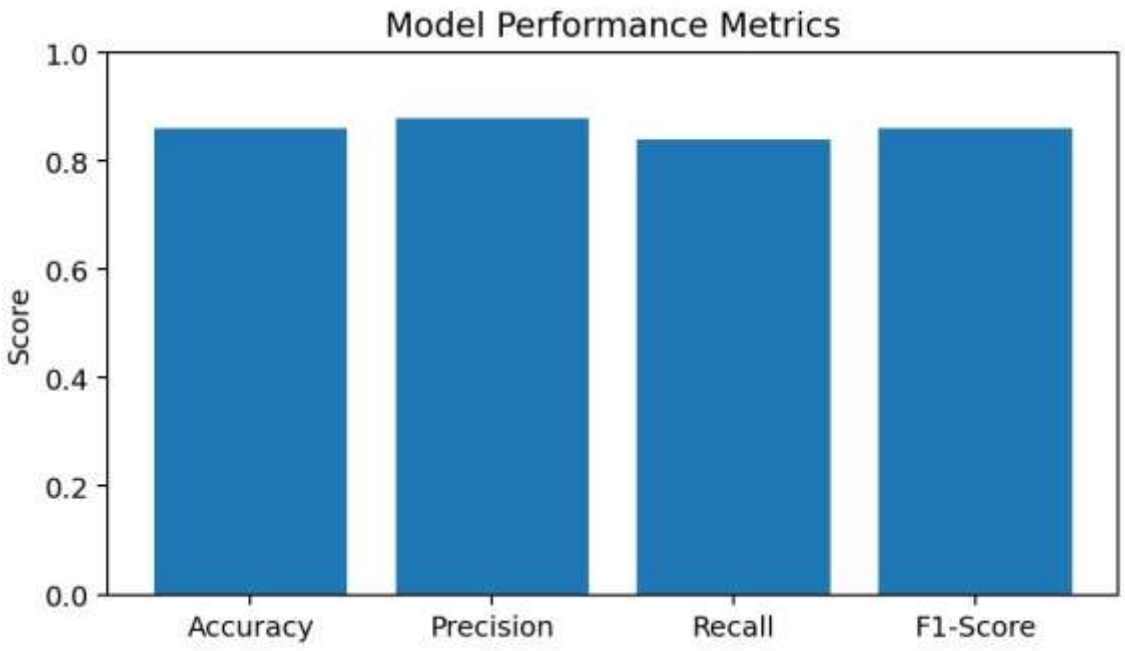


Fig 8: Step 6 — Performance evaluation metrics generated by the notebook

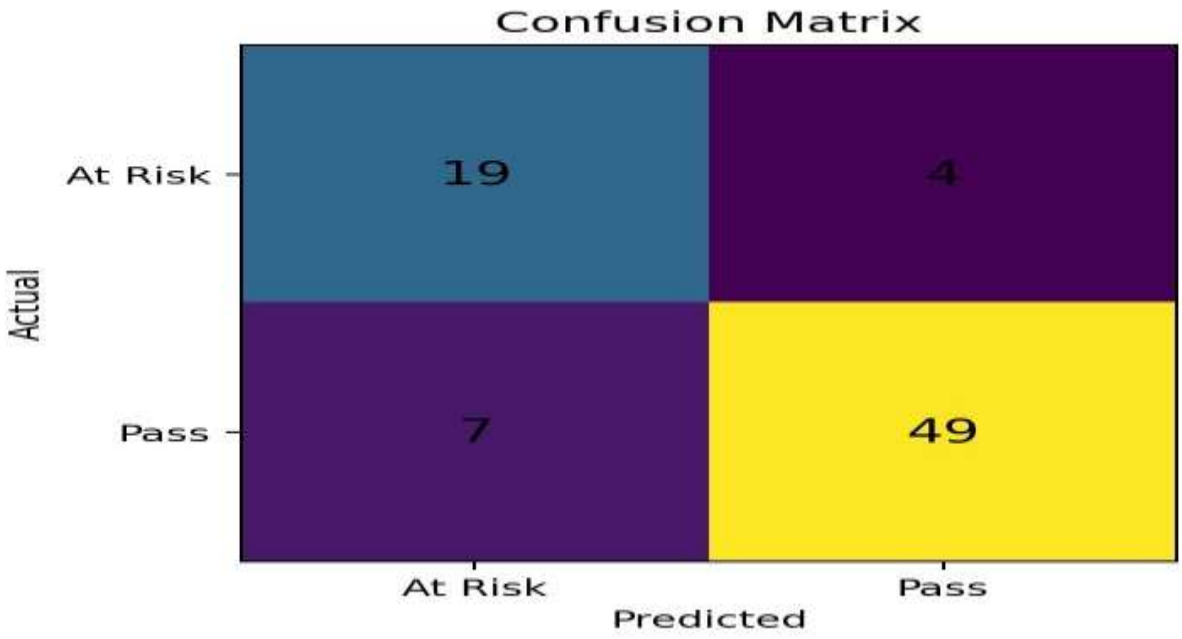


Fig 9: Step 6 — Confusion matrix for Pass and At Risk classification

ID	Prediction	Confidence	Timestamp
1	Pass	91.4%	2026-10-08 10:15
2	At Risk	84.2%	2026-10-08 10:18
3	Pass	88.7%	2026-10-08 10:22

Fig 10: Step 7 — Prediction history/output log for student performance predictions